React map() Practice Questions

Total Questions: 10
Difficulty: 2 Easy + 4 Medium + 4 Hard

Instructions

1. Create a functional React component for each question.
2. Use `map()` to display arrays dynamically.
3. Use a unique `key` for every mapped element.
4. Use JSX to display the data.
5. Use `filter()` and `reduce()` wherever specified.
6. Apply basic CSS wherever required.

EASY QUESTIONS (2)

Q1. Display a List of Student Names

Given Data:

```
const students = ["Rahul", "Priya", "Amit", "Neha", "Karan"];
```

Task:

Create a component named `StudentList` that displays all student names inside an unordered list using `map()`.

Expected Output:

- Rahul
- Priya
- Amit
- Neha
- Karan

Requirements:

- Use `map()` to display the names.
- Use `<ul>` and `<li>`.
- Assign a unique `key` to every list item.

Q2. Display Products as Cards

Given Data:

```
const products = [
  { id: 1, name: "Laptop", price: 50000 },
  { id: 2, name: "Mouse", price: 500 },
  { id: 3, name: "Keyboard", price: 1200 },
  { id: 4, name: "Monitor", price: 10000 }
];
```

Task:

Create a component named `ProductList` that displays every product inside a separate card.

Each card must display:

- Product name
- Product price
- A button labelled "Buy Now"

Requirements:

- Use `map()` to generate the cards.
- Use `product.id` as the key.
- Apply CSS to add borders, padding, spacing, and a background color.
- Display prices using the ₹ symbol.

MEDIUM QUESTIONS (4)

Q3. Student Marks Report

Given Data:

```
const students = [
  { id: 1, name: "Rahul", marks: 85 },
  { id: 2, name: "Priya", marks: 32 },
  { id: 3, name: "Amit", marks: 67 },
  { id: 4, name: "Neha", marks: 91 },
  { id: 5, name: "Karan", marks: 28 }
];
```

Task:

Create a component named `MarksReport` that displays each student's name, marks, and result.

Rules:

- Marks greater than or equal to 35: Pass
- Marks below 35: Fail

- Display Pass in green.
- Display Fail in red.

Requirements:

- Use `map()`.
- Use the ternary operator for conditional rendering.
- Use a unique key for each student.

Q4. Display Affordable Products Using filter()

Given Data:

```
const products = [
  { id: 1, name: "Laptop", price: 50000 },
  { id: 2, name: "Mouse", price: 500 },
  { id: 3, name: "Keyboard", price: 1200 },
  { id: 4, name: "Headphones", price: 2500 },
  { id: 5, name: "USB Cable", price: 200 }
];
```

Task:

Create a component named `AffordableProducts` that displays only products costing less than ₹2,000.

Requirements:

- Use `filter()` to select products.
- Use `map()` to display the filtered products.
- Display the product name and price.
- Do not display products costing ₹2,000 or more.

Expected Products:

- Mouse
- Keyboard
- USB Cable

Q5. Display Employee Details in a Table

Given Data:

```
const employees = [
  { id: 1, name: "Aarav", department: "IT", salary: 45000 },
  { id: 2, name: "Diya", department: "HR", salary: 35000 },
  { id: 3, name: "Rohan", department: "IT", salary: 55000 },
```

```
  { id: 4, name: "Meera", department: "Sales", salary: 40000 }
];
```

Task:

Create a component named `EmployeeTable` that displays employee details in an HTML table.

Table Columns:

- ID
- Name
- Department
- Salary

Requirements:

- Use `map()` to generate table rows.
- Use the employee ID as the key.
- Display salary using the ₹ symbol.
- Include `<thead>` and `<tbody>`.
- Add basic CSS to style the table.

Q6. Shopping Cart with Subtotals and Grand Total

Given Data:

```
const cart = [
  { id: 1, name: "Notebook", price: 50, quantity: 3 },
  { id: 2, name: "Pen", price: 10, quantity: 5 },
  { id: 3, name: "Bag", price: 800, quantity: 1 },
  { id: 4, name: "Bottle", price: 200, quantity: 2 }
];
```

Task:

Create a component named `ShoppingCart` that displays each item's name, price, quantity, and subtotal.

Requirements:

- Use `map()` to display cart items.
- Calculate each subtotal using `price * quantity`.
- Use `reduce()` to calculate the grand total.
- Display the grand total separately.

Expected Subtotals:

- Notebook: ₹150
- Pen: ₹50

- Bag: ₹800
- Bottle: ₹400

Expected Grand Total: ₹1,400

HARD QUESTIONS (4)

Q7. Display Categories and Products Using Nested map()

Given Data:

```
const categories = [
  {
    id: 1,
    name: "Electronics",
    products: [
      { id: 101, name: "Laptop", price: 50000 },
      { id: 102, name: "Mouse", price: 500 }
    ]
  },
  {
    id: 2,
    name: "Clothing",
    products: [
      { id: 201, name: "T-Shirt", price: 700 },
      { id: 202, name: "Jeans", price: 1500 }
    ]
  },
  {
    id: 3,
    name: "Accessories",
    products: [
      { id: 301, name: "Watch", price: 2500 },
      { id: 302, name: "Wallet", price: 900 }
    ]
  }
];
```

Task:

Create a component named `CategoryList` that displays each category and its products.

Requirements:

- Use an outer `map()` to display categories.
- Use an inner `map()` to display products.
- Display category names as headings.
- Display product names and prices below each heading.
- Use stable keys for categories and products.

- Apply basic CSS to separate the categories visually.

Q8. Filter Top Students and Calculate Average Marks

Given Data:

```
const students = [
  { id: 1, name: "Rahul", marks: 85 },
  { id: 2, name: "Priya", marks: 32 },
  { id: 3, name: "Amit", marks: 67 },
  { id: 4, name: "Neha", marks: 91 },
  { id: 5, name: "Karan", marks: 28 },
  { id: 6, name: "Isha", marks: 76 }
];
```

Task:

Create a component named `TopStudents` that displays students who scored at least 60 marks.

Requirements:

1. Use `filter()` to select students with marks greater than or equal to 60.
2. Use `map()` to display their names and marks.
3. Use `reduce()` to calculate their total marks.
4. Calculate their average marks.
5. Display the number of selected students.
6. Handle the case where no students qualify to avoid division by zero.

Expected Results:

- Selected students: Rahul, Amit, Neha, Isha
- Total marks: 319
- Average marks: 79.75
- Number of selected students: 4

Q9. Order Dashboard with Nested Mapping

Given Data:

```
const orders = [
  {
    id: 1,
    customer: "Rahul",
    items: [
      { id: 11, name: "Laptop", price: 50000, quantity: 1 },
      { id: 12, name: "Mouse", price: 500, quantity: 2 }
    ]
```

```
    },
    {
      id: 2,
      customer: "Priya",
      items: [
        { id: 21, name: "Book", price: 300, quantity: 3 },
        { id: 22, name: "Bag", price: 1000, quantity: 1 }
      ]
    },
    {
      id: 3,
      customer: "Amit",
      items: [
        { id: 31, name: "Keyboard", price: 1500, quantity: 1 },
        { id: 32, name: "Monitor", price: 8000, quantity: 1 }
      ]
    }
  ];
```

Task:

Create a component named `OrderDashboard` that displays every customer's order.

Each order must show:

- Customer name
- Item names
- Item prices
- Quantities
- Item subtotals
- Order total

Requirements:

- Use an outer `map()` for orders.
- Use an inner `map()` for items.
- Use `reduce()` to calculate each order's total.
- Calculate and display the grand total of all orders.
- Use stable keys.
- Display each order inside a separate card.

Expected Order Totals:

- Rahul: ₹51,000
- Priya: ₹1,900
- Amit: ₹9,500

Expected Grand Total: ₹62,400

Q10. Product Analytics Dashboard

Given Data:

```
const products = [
  { id: 1, name: "Laptop", category: "Electronics", price: 50000, stock: 5 },
  { id: 2, name: "Mouse", category: "Electronics", price: 500, stock: 20 },
  { id: 3, name: "Keyboard", category: "Electronics", price: 1500, stock: 0 },
  { id: 4, name: "T-Shirt", category: "Clothing", price: 700, stock: 15 },
  { id: 5, name: "Jeans", category: "Clothing", price: 1500, stock: 3 },
  { id: 6, name: "Watch", category: "Accessories", price: 2500, stock: 0 }
];
```

Task:

Create a component named `ProductAnalytics` that displays product information and inventory statistics.

Part A: Product List

Use `map()` to display:

- Product name
- Category
- Price
- Stock quantity
- Availability status

Display `In Stock` when stock is greater than zero. Otherwise, display `Out of Stock`.

Part B: Filter Products

Use `filter()` to display only products that are in stock.

Part C: Inventory Calculations

Use `reduce()` to calculate:

- Total units in stock
- Total inventory value using `price * stock`

Also display the number of products that are out of stock.

Part D: Styling

Display the products in cards or a table. Use different text colors for available and unavailable products.

Expected Results:

- In-stock products: Laptop, Mouse, T-Shirt, Jeans
- Total units in stock: 43
- Total inventory value: ₹2,89,000
- Out-of-stock products: 2

Submission Checklist

Before submitting, verify that:

1. All 10 questions are completed.
2. `map()` is used correctly to render lists.
3. Every mapped element has a suitable key.
4. `filter()` is used wherever required.
5. `reduce()` calculations are correct.
6. Conditional rendering works correctly.
7. The expected results match the output.
8. The JSX and CSS are valid.

Important: Do not copy the same component for every question. Create separate components and practise each problem independently.